

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 35%

Dr.G.Ayyappan

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	5
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	5
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	5
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	5
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	5
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	5
7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	5

8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	5
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	5
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	5

Code of Conduct:

*I Abishana S K (192511242) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1) Assume the relations are:

EMPLOYEE:

EmpID	Name	DeptID	Salary
101	Rahul	D1	60000
102	Anita	D2	45000
103	Karan	D1	70000
104	Sneha	D3	50000

DEPARTMENT:

DeptID	DeptName
D1	IT
D2	HR
D3	Finance

(a) Selection (σ)

Retrieve employees earning more than ₹50,000.

Result

EmpID	Name	DeptID	Salary
101	Rahul	D1	60000
103	Karan	D1	70000

Explanation:

The **Selection (σ)** operator filters rows based on a specified condition. Here, only employees with a salary greater than ₹50,000 are selected.

(b) Projection (π)

Retrieve only employee names.

Result

Name

Rahul

Anita

Karan

Sneha

Explanation:

The **Projection (π)** operator retrieves only the required column while eliminating duplicate values.

(c) Union ($\cup$)

Assume another relation:

TEMP_EMPLOYEE

Name

Ravi

Name

Anita

Expression:

Result**Name**

Rahul

Anita

Karan

Sneha

Ravi

Explanation:

The **Union (U)** operation combines tuples from two compatible relations and removes duplicate records.

(d) Difference (-)

Find employees who are not temporary employees.

Result**Name**

Rahul

Karan

Sneha

Explanation:

The **Difference (-)** operator returns tuples that exist in the first relation but not in the second relation.

(e) Cartesian Product (×)**Result**

Every employee is paired with every department.

Number of tuples:

Example rows:

Name DeptName

Rahul IT

Rahul HR

Rahul Finance

Anita IT

... ..

Explanation:

The **Cartesian Product (×)** combines every tuple of one relation with every tuple of another relation.

(f) Join (⋈)

Retrieve employee names along with their department names.

Result

EmpID	Name	DeptName	Salary
101	Rahul	IT	60000
102	Anita	HR	45000
103	Karan	IT	70000
104	Sneha	Finance	50000

Explanation:

The Join (⋈) operation combines related tuples from both relations based on the common attribute DeptID. This produces meaningful information by displaying employee details together with their department names.

Final Interpretation

Using relational algebra operators such as Selection, Projection, Union, Difference, Cartesian Product, and Join allows efficient retrieval and manipulation of relational data. Among these operations, Join is one of the most frequently used because it combines related information from multiple tables, making database queries more meaningful and practical.

2) Before creating the **STUDENT** table, the **DEPARTMENT** table should be created because DeptID in the STUDENT table acts as a foreign key.

Create the DEPARTMENT Table

```
CREATE TABLE DEPARTMENT (  
    DeptID INT PRIMARY KEY,  
    DeptName VARCHAR(50) NOT NULL UNIQUE  
);
```

Explanation:

- DeptID is the Primary Key, ensuring each department has a unique ID.
- DeptName is marked as NOT NULL so every department must have a name.
- UNIQUE prevents duplicate department names.

Create the STUDENT Table

```
CREATE TABLE STUDENT (  
    SID INT PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    DOB DATE NOT NULL,  
    DeptID INT,  
    CONSTRAINT FK_Student_Department  
        FOREIGN KEY (DeptID)  
        REFERENCES DEPARTMENT(DeptID)  
    ON UPDATE CASCADE
```

ON DELETE SET NULL
);

Explanation:

- SID is the Primary Key, uniquely identifying each student.
- Name is declared as NOT NULL, ensuring every student record has a name.
- DOB is stored using the DATE data type and cannot be left empty.
- DeptID is a Foreign Key that references the DeptID in the DEPARTMENT table.
- ON UPDATE CASCADE automatically updates the foreign key value if the department ID changes.
- ON DELETE SET NULL sets the student's DeptID to NULL if the referenced department is deleted, thereby maintaining referential integrity.

Sample Data Insertion:

```
INSERT INTO DEPARTMENT (DeptID, DeptName)
VALUES
(101, 'Computer Science'),
(102, 'Information Technology'),
(103, 'Electronics');
INSERT INTO STUDENT (SID, Name, DOB, DeptID)
VALUES
(1, 'Rahul', '2004-05-12', 101),
(2, 'Anita', '2003-11-20', 102),
(3, 'Karan', '2004-08-15', 101);
```

Interpretation

The above DDL statements create two normalized tables with appropriate constraints to ensure data integrity. The Primary Key uniquely identifies each record, the Foreign Key maintains the relationship between students and departments, while the NOT NULL and UNIQUE constraints help enforce valid and consistent data. This design reduces redundancy and supports efficient database management.

3) Problem Understanding

The objective is to analyze the SALES table by grouping records based on the product. Aggregate functions are used to calculate summary statistics, while the HAVING clause filters grouped results according to a specified condition.

Given SALES Table

SaleID	Product	Region	Amount
1	Laptop	North	50000
2	Laptop	South	45000
3	Mobile	North	20000
4	Mobile	South	25000

5	Laptop	North	55000
6	Tablet	East	30000
7	Tablet	East	35000

SQL Query

```
SELECT Product,
       COUNT(*) AS TotalSales,
       SUM(Amount) AS TotalRevenue,
       AVG(Amount) AS AverageSale,
       MAX(Amount) AS HighestSale,
       MIN(Amount) AS LowestSale
FROM SALES
GROUP BY Product
HAVING AVG(Amount) > 25000;
```

Result

Product	TotalSales	TotalRevenue	AverageSale	HighestSale	LowestSale
Laptop	3	150000	50000	55000	45000
Tablet	2	65000	32500	35000	30000

Solution Explanation

1. The GROUP BY Product clause groups all sales records according to the product name.
2. COUNT(*) counts the number of sales made for each product.
3. SUM(Amount) calculates the total revenue earned by each product.
4. AVG(Amount) computes the average sales amount.
5. MAX(Amount) identifies the highest sale value.
6. MIN(Amount) identifies the lowest sale value.
7. The HAVING AVG(Amount) > 25000 clause filters the grouped results and displays only those products whose average sales exceed ₹25,000.

Interpretation of Results

The query shows that Laptop generated the highest revenue with an average sale of ₹50,000, while Tablet recorded an average sale of ₹32,500. The Mobile category is excluded because its average sales amount (₹22,500) does not satisfy the HAVING condition. This analysis helps identify products with stronger sales performance.

4) Problem Understanding

The objective is to identify employees whose salary is greater than the average salary of the department in which they work. Since the average salary must be calculated separately for each employee's department, a correlated sub-query is the

appropriate solution. The inner query depends on the current row processed by the outer query.

Assume the EMPLOYEE Table

EmpID	EmpName	DeptID	Salary
101	Rahul	D1	60000
102	Anita	D1	50000
103	Karan	D2	70000
104	Sneha	D2	55000
105	Priya	D3	45000
106	Arjun	D3	40000

SQL Query

```
SELECT EmpID, EmpName, DeptID, Salary
FROM EMPLOYEE E
WHERE Salary >
(
    SELECT AVG(Salary)
    FROM EMPLOYEE
    WHERE DeptID = E.DeptID
);
```

Result

EmpID	EmpName	DeptID	Salary
101	Rahul	D1	60000
103	Karan	D2	70000
105	Priya	D3	45000

Solution Explanation

1. The outer query retrieves the employee ID, employee name, department ID, and salary from the **EMPLOYEE** table.
2. The inner query calculates the **average salary** of the department corresponding to the current employee.
3. The condition DeptID = E.DeptID makes the inner query **correlated** because it references the outer query's current row.
4. The WHERE Salary > (...) condition compares each employee's salary with the average salary of their own department.
5. Only employees whose salary exceeds their departmental average are included in the final result.

Accuracy Verification

Department-wise average salaries:

- **D1:** $(60000 + 50000) \div 2 = 55,000$

- **D2:** $(70000 + 55000) \div 2 = \mathbf{62,500}$
- **D3:** $(45000 + 40000) \div 2 = \mathbf{42,500}$

Comparison:

Employee	Salary	Department Average	Result
Rahul	60000	55000	Selected
Anita	50000	55000	Not Selected
Karan	70000	62500	Selected
Sneha	55000	62500	Not Selected
Priya	45000	42500	Selected
Arjun	40000	42500	Not Selected

The output is therefore correct.

Interpretation of Results

The correlated sub-query compares each employee's salary with the average salary of their own department. Employees Rahul, Karan, and Priya earn more than the average salary in their respective departments and are therefore returned by the query. This type of query is useful for performance analysis, salary benchmarking, and identifying above-average earners within each department.

5) Problem Understanding

The objective is to demonstrate the use of different SQL JOIN operations to retrieve related data from the EMPLOYEE and PROJECT tables. Each JOIN type returns data differently depending on whether matching records exist in both tables.

Assume the Tables

EMPLOYEE

EmpID	EmpName	ProjectID
101	Rahul	P1
102	Anita	P2
103	Karan	P3
104	Sneha	NULL

PROJECT

ProjectID	ProjectName
P1	Hospital Management
P2	Online Banking
P4	E-Commerce System

INNER JOIN:

SQL Query

```
SELECT E.EmpID, E.EmpName, P.ProjectName
FROM EMPLOYEE E
INNER JOIN PROJECT P
```

ON E.ProjectID = P.ProjectID;

Result

EmpID	EmpName	ProjectName
101	Rahul	Hospital Management
102	Anita	Online Banking

Explanation

The INNER JOIN returns only those records where a matching ProjectID exists in both tables.

LEFT OUTER JOIN:

SQL Query

```
SELECT E.EmpID, E.EmpName, P.ProjectName
FROM EMPLOYEE E
LEFT OUTER JOIN PROJECT P
ON E.ProjectID = P.ProjectID;
```

Result

EmpID	EmpName	ProjectName
101	Rahul	Hospital Management
102	Anita	Online Banking
103	Karan	NULL
104	Sneha	NULL

Explanation

The LEFT OUTER JOIN returns all employees. If an employee has no matching project, the project details are displayed as NULL.

RIGHT OUTER JOIN:

SQL Query

```
SELECT E.EmpID, E.EmpName, P.ProjectName
FROM EMPLOYEE E
RIGHT OUTER JOIN PROJECT P
ON E.ProjectID = P.ProjectID;
```

Result

EmpID	EmpName	ProjectName
101	Rahul	Hospital Management
102	Anita	Online Banking
NULL	NULL	E-Commerce System

Explanation

The RIGHT OUTER JOIN returns all projects. If a project has no assigned employee, the employee details appear as NULL.

FULL OUTER JOIN:

SQL Query

```
SELECT E.EmpID, E.EmpName, P.ProjectName
FROM EMPLOYEE E
FULL OUTER JOIN PROJECT P
ON E.ProjectID = P.ProjectID;
```

Result

EmpID	EmpName	ProjectName
101	Rahul	Hospital Management
102	Anita	Online Banking
103	Karan	NULL
104	Sneha	NULL
NULL	NULL	E-Commerce System

Explanation

The FULL OUTER JOIN returns all records from both tables. Matching records are combined, while non-matching records contain NULL values for the missing side.

CROSS JOIN:

SQL Query

```
SELECT E.EmpName, P.ProjectName
FROM EMPLOYEE E
CROSS JOIN PROJECT P;
```

Result

Since there are 4 employees and 3 projects:

Total rows = $4 \times 3 = 12$

Example output:

EmpName	ProjectName
Rahul	Hospital Management
Rahul	Online Banking
Rahul	E-Commerce System
Anita	Hospital Management
Anita	Online Banking
...	...

Explanation

The CROSS JOIN produces the Cartesian product by combining every employee with every project, regardless of any relationship between them.

Interpretation of Results

Different JOIN operations are used to retrieve data based on different requirements. The INNER JOIN returns only matching records, LEFT OUTER JOIN includes all employees, RIGHT OUTER JOIN includes all projects, FULL OUTER JOIN combines all records from both tables, and CROSS JOIN generates every possible employee-project combination. These JOIN operations are essential for retrieving

related information from multiple tables and are widely used in relational database management systems.

6) Problem Understanding

The objective is to create a SQL view that summarizes employee information for each department. The view should display the department name, the total number of employees, and the average salary. After creating the view, a query should be written to retrieve data from it. A view simplifies complex queries by storing them as a virtual table.

Assume the Tables

DEPARTMENT

DeptID	DeptName
D1	IT
D2	HR
D3	Finance

EMPLOYEE

EmpID	EmpName	DeptID	Salary
101	Rahul	D1	60000
102	Anita	D1	50000
103	Karan	D2	70000
104	Sneha	D2	55000
105	Priya	D3	45000

SQL Statement to Create the View

```
CREATE VIEW DeptSalaryView AS
SELECT
    D.DeptName,
    COUNT(E.EmpID) AS TotalEmployees,
    AVG(E.Salary) AS AverageSalary
FROM DEPARTMENT D
JOIN EMPLOYEE E
ON D.DeptID = E.DeptID
GROUP BY D.DeptName;
```

Query on the View

```
SELECT *
FROM DeptSalaryView;
```

Result

DeptName	TotalEmployees	AverageSalary
IT	2	55000
HR	2	62500

Finance	1	45000
---------	---	-------

Solution Explanation

1. The CREATE VIEW statement creates a virtual table named DeptSalaryView.
2. The JOIN operation combines the DEPARTMENT and EMPLOYEE tables using the common attribute DeptID.
3. The COUNT(E.EmpID) function calculates the total number of employees in each department.
4. The AVG(E.Salary) function calculates the average salary of employees in each department.
5. The GROUP BY D.DeptName clause groups records department-wise.
6. The SELECT * FROM DeptSalaryView statement retrieves all records stored in the view.

Accuracy Verification

Department-wise calculations:

- IT: Employees = 2, Average Salary = $(60000 + 50000) \div 2 = 55,000$
- HR: Employees = 2, Average Salary = $(70000 + 55000) \div 2 = 62,500$
- Finance: Employees = 1, Average Salary = 45,000

The output is therefore correct.

Interpretation of Results

The DeptSalaryView provides a summarized view of employee information for each department. It displays the department name, the total number of employees, and the average salary without requiring the user to repeatedly write complex JOIN and aggregate queries. Views improve query simplicity, enhance data security by exposing only required information, and make database reporting more efficient.

7) Problem Understanding

The objective is to identify the students who are enrolled in every course offered by the institution. Since the query requires finding students who satisfy a condition for **all** courses, the relational algebra **division** ($\div$) operator is the most suitable choice. Division is specifically used to answer "for all" type queries.

Assume the Relations

STUDENT

SID	Name
101	Rahul
102	Anita
103	Karan

COURSE

CourseID	CourseName
C1	Database Management
C2	Operating Systems
C3	Computer Networks

ENROLLMENT

SID	CourseID
101	C1
101	C2
101	C3
102	C1
102	C2
103	C1
103	C2
103	C3

Relational Algebra Expression

Step 1: Find students enrolled in all courses

This returns the SID of students who are enrolled in every course.

Step 2: Retrieve the student names

Result:

Name
Rahul
Karan

Solution Explanation

1. The COURSE relation contains the list of all courses offered.
2. The ENROLLMENT relation stores the courses taken by each student.
3. The division ($\div$) operator compares the enrollment records with the complete list of courses and identifies students who have enrolled in every course.
4. The resulting student IDs are joined with the STUDENT relation to obtain the corresponding student names.
5. Finally, the projection (π) operator displays only the Name attribute.

Accuracy Verification

Enrollment details:

- Rahul is enrolled in C1, C2, and C3.
- Anita is enrolled only in C1 and C2.
- Karan is enrolled in C1, C2, and C3.

Since Rahul and Karan are enrolled in all the courses offered, they satisfy the given condition.

Interpretation of Results

The division operator is the most appropriate relational algebra operation for solving queries that require satisfying a condition for every value in another relation. In this example, it identifies students who have enrolled in all available courses. Such queries are commonly used in educational database systems to analyze complete course participation and student enrollment patterns.

8) Problem Understanding

The objective is to demonstrate the use of SQL Data Manipulation Language (DML) statements, namely INSERT, UPDATE, and DELETE, while maintaining database integrity. The operations should follow the constraints defined in the database, such as primary key and foreign key constraints, to ensure that the data remains valid and consistent.

Assume the Tables

DEPARTMENT

DeptID	DeptName
101	Computer Science
102	Information Technology

STUDENT

SID	Name	DOB	DeptID
1	Rahul	2004-05-12	101
2	Anita	2003-11-20	102

1. INSERT Statement

Insert a new student into the STUDENT table.

```
INSERT INTO STUDENT (SID, Name, DOB, DeptID)
VALUES (3, 'Karan', '2004-08-15', 101);
```

Explanation

This statement inserts a new student record. Since DeptID = 101 already exists in the DEPARTMENT table, the foreign key constraint is satisfied and the insertion is successful.

2. UPDATE Statement

Update the department of student with SID = 3.

```
UPDATE STUDENT
SET DeptID = 102
WHERE SID = 3;
```

Explanation

This statement changes the student's department from Computer Science to Information Technology. The update is valid because DeptID = 102 exists in the DEPARTMENT table, maintaining referential integrity.

3. DELETE Statement

Delete the student with SID = 3.

```
DELETE FROM STUDENT
```

```
WHERE SID = 3;
```

Explanation

This statement removes the student record with SID = 3. Since only the student record is deleted and no integrity constraints are violated, the deletion is completed successfully.

Integrity Constraint Enforcement

The following statement will fail because DeptID = 105 does not exist in the DEPARTMENT table.

```
INSERT INTO STUDENT (SID, Name, DOB, DeptID)
```

```
VALUES (4, 'Priya', '2005-02-10', 105);
```

Reason: The foreign key constraint prevents inserting a student into a department that does not exist, thereby preserving data integrity.

Solution Explanation

1. The INSERT statement adds a new student record while satisfying the foreign key constraint.
2. The UPDATE statement modifies the department of an existing student without violating referential integrity.
3. The DELETE statement removes a student record safely from the table.
4. An attempt to insert a record with an invalid DeptID is rejected because of the foreign key constraint.

Interpretation of Results

The INSERT, UPDATE, and DELETE statements are essential DML operations used to manage data in a relational database. Integrity constraints such as primary keys and foreign keys ensure that only valid data is inserted or modified. This helps maintain data consistency, prevents invalid references, and improves the reliability of the database system.

9) Problem Understanding

The objective is to write a Tuple Relational Calculus (TRC) expression to retrieve employees who satisfy two conditions: they must belong to the IT department and their salary must be greater than ₹50,000. Tuple Relational Calculus is a non-procedural query language, where the required result is specified without describing the steps to retrieve it.

Assume the Relations

EMPLOYEE

EmpID	EmpName	DeptID	Salary
101	Rahul	D1	60000
102	Anita	D2	45000
103	Karan	D1	70000
104	Sneha	D3	50000

DEPARTMENT

DeptID	DeptName
D1	IT
D2	HR
D3	Finance

Tuple Relational Calculus Expression

Result

EmpID	EmpName	DeptID	Salary
101	Rahul	D1	60000
103	Karan	D1	70000

Solution Explanation

1. EMPLOYEE(e) specifies that e is a tuple from the EMPLOYEE relation.
2. DEPARTMENT(d) specifies that d is a tuple from the DEPARTMENT relation.
3. e.DeptID = d.DeptID matches each employee with the corresponding department.
4. d.DeptName = 'IT' selects only employees working in the IT department.
5. e.Salary > 50000 filters employees whose salary is greater than ₹50,000.
6. The expression returns all employee tuples that satisfy both conditions.

Accuracy Verification

Checking the employee records:

- Rahul works in the IT department and earns ₹60,000. correct
- Anita works in the HR department and earns ₹45,000. wrong
- Karan works in the IT department and earns ₹70,000. correct
- Sneha works in the Finance department and earns ₹50,000. wrong

Therefore, the employees satisfying the given conditions are Rahul and Karan.

Interpretation of Results

The Tuple Relational Calculus expression retrieves employees who belong to the IT department and have a salary greater than ₹50,000 without specifying the sequence of operations. It focuses on the conditions that the required tuples must satisfy, making it a declarative approach to querying relational databases. This method is

useful for expressing complex conditions in database theory and relational query processing.

10) Problem Understanding

The objective is to design a relational database schema for an Airline Reservation System and demonstrate how SQL queries can be used to retrieve meaningful information. The schema should establish relationships between tables using primary and foreign keys. The queries should illustrate the use of joins, sub-queries, and views.

Relational Schema

PASSENGER

Attribute	Description
PassengerID (PK)	Unique passenger ID
PassengerName	Name of the passenger
Phone	Contact number

FLIGHT

Attribute	Description
FlightID (PK)	Unique flight ID
FlightName	Flight name
Source	Departure city
Destination	Arrival city

BOOKING

Attribute	Description
BookingID (PK)	Unique booking ID
PassengerID (FK)	References PASSENGER
FlightID (FK)	References FLIGHT
JourneyDate	Date of travel
SeatNo	Seat number

Relationship:

- One passenger can have many bookings.
- One flight can have many bookings.
- The BOOKING table connects the PASSENGER and FLIGHT tables through foreign keys.

SQL Query 1: INNER JOIN

Retrieve passenger names along with their booked flight names.

```
SELECT P.PassengerName, F.FlightName
FROM PASSENGER P
JOIN BOOKING B
ON P.PassengerID = B.PassengerID
JOIN FLIGHT F
ON B.FlightID = F.FlightID;
```

SQL Query 2: LEFT OUTER JOIN

Display all passengers, including those who have not booked any flight.

```
SELECT P.PassengerName, F.FlightName
FROM PASSENGER P
LEFT JOIN BOOKING B
ON P.PassengerID = B.PassengerID
LEFT JOIN FLIGHT F
ON B.FlightID = F.FlightID;
```

SQL Query 3: Sub-query

Retrieve passengers who have booked Flight ID 'F101'.

```
SELECT PassengerName
FROM PASSENGER
WHERE PassengerID IN
(
    SELECT PassengerID
    FROM BOOKING
    WHERE FlightID = 'F101'
);
```

SQL Query 4: Create a View

Create a view displaying passenger names and their flight details.

```
CREATE VIEW PassengerFlightView AS
SELECT P.PassengerName,
       F.FlightName,
       F.Source,
       F.Destination
FROM PASSENGER P
JOIN BOOKING B
ON P.PassengerID = B.PassengerID
JOIN FLIGHT F
ON B.FlightID = F.FlightID;
```

SQL Query 5: Query on the View

Display all records from the view.

```
SELECT *
FROM PassengerFlightView;
```

Solution Explanation

1. The relational schema consists of three tables: PASSENGER, FLIGHT, and BOOKING.
2. The BOOKING table establishes relationships between passengers and flights using foreign keys.

3. The first query uses an INNER JOIN to display passengers and their booked flights.
4. The second query uses a LEFT OUTER JOIN to include passengers who have not made any booking.
5. The third query uses a sub-query to identify passengers booked on a specific flight.
6. The fourth query creates a view to simplify repeated access to passenger and flight information.
7. The fifth query retrieves the data stored in the view.

Accuracy Verification

The schema is properly normalized, with primary keys uniquely identifying records and foreign keys maintaining referential integrity. The SQL queries correctly demonstrate the use of joins, a sub-query, and a view, satisfying the requirements of the Airline Reservation System.

Interpretation of Results

The proposed relational schema efficiently stores passenger, flight, and booking information while reducing data redundancy. The SQL queries demonstrate different techniques for retrieving and organizing data. Joins combine information from multiple tables, sub-queries retrieve records based on specific conditions, and views simplify complex queries for repeated use. This design provides a structured and efficient solution for managing an Airline Reservation System.